

Примечание: Этот канвас — главный источник правды. Даже если часть переписки выпадет из памяти чата, все ключевые идеи, архитектура и договорённости будут зафиксированы здесь и не потеряются.

Проект Snake Groove (рабочее название)

0. Motivation, Principles & Learning

Motivation during the process

- Каждый шаг проекта связываем с большой картиной развития Snake Groove.
- Подчёркиваем личные выгоды: освоение новых инструментов, практический опыт, улучшение архитектуры.
- Фиксируем маленькие победы: каждый завершённый шаг = осязаемый прогресс.
- Цель: поддерживать интерес и вдохновение, чтобы проект приносил радость.

Principles

Motivation

- Создать игру **Snake Groove** — классическая механика + музыка и мультяшная графика.
- Практиковаться в архитектуре, Clean Code и паттернах.
- Освоить современные технологии (.NET 8, Unity, Blazor, CI/CD, GitHub, Copilot, Cursor).
- Сделать проект с потенциалом роста и монетизации (скины, музыка, внутриигровая валюта).

Principles

- **Чистая архитектура:** Core.GameLogic + Platform Adapters.
- **Clean Code:** SOLID, KISS, DRY, YAGNI.
- **Расширяемость:** закладываем точки расширения с `// TODO`.
- **Устойчивость:** Global Exception Handling, Serilog.
- **CI/CD и контроль версий:** GitHub, GitHub Actions, Kanban (GitHub Projects).
- **Кроссплатформенность:** Unity (Win/Mobile/Web), Blazor WASM (web demo).
- **Будущая монетизация:** предусмотреть возможность добавления скинов, музыки и валюты.

1. Vision & Core Loop

- **Идея:** Аркадный Snake с музыкальным твистом — каждая съеденная «еда» добавляет новый аудиотрек, змейка двигается в ритме.
- **Цель MVP (Phase 1):** Классический Snake с расширяемой архитектурой (см. TODO-метки).
- **Будущее расширение:** Разные местности (горы, леса, вода и др.), где каждая влияет на геймплей:
- Горы: замедление движения, эффект эха в музыке.

- Леса: визуальные фильтры, дополнительные перкуссионные слои.
- Вода: скольжение/ускорение, приглушение звуков.
- Эти элементы делают каждое прохождение уникальным и разнообразным.
- **Идея:** Аркадный Snake с музыкальным твистом — каждая съеденная «еда» добавляет новый аудиотрек, змейка двигается в ритме.
- **Цель MVP (Phase 1):** Классический Snake с расширяемой архитектурой (см. TODO-метки).

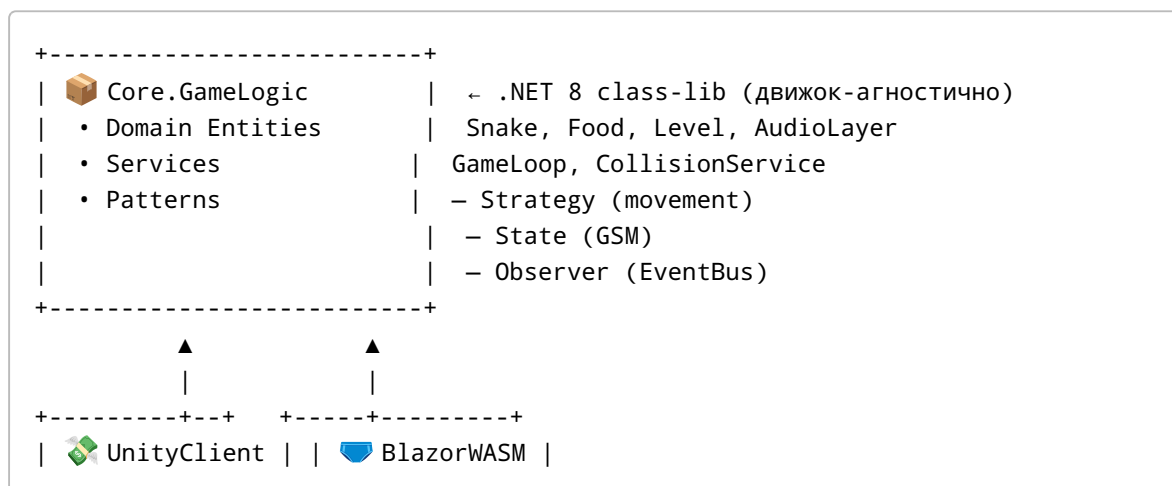
2. Milestone Roadmap

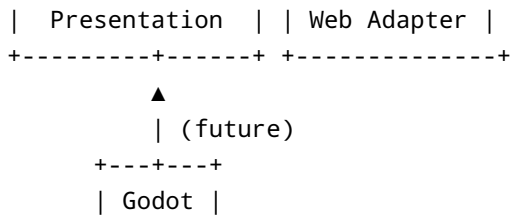
Организация работы по фазам

- Каждая фаза (Phase 0, Phase 1 и т.д.) может вестись в отдельном чате для удобства.
- Главный контекст, архитектура и ключевые договорённости фиксируются в этом канвасе.
- Этот документ служит источником правды (Single Source of Truth).
- Из каждого чата-фазы итоговые выводы и решения переносятся сюда для консолидации.
- Для работы в новом чате добавляется ссылка на этот канвас, чтобы главный контекст был доступен всегда.

Phase	Веха	Платформы	Ключ фичи
0	Project Setup	n/a	Repo, CI/CD, базовый скелет Core .GameLogic
1	Classic Snake MVP	Desktop Win + WebGL	Меню, пауза, настройки размера поля
2	Music Layers	+Mobile	Генеративная музыка, аудио-миксер
3	Scrolling Playfield	All	Динамическая камера, диагонали, прыжки
4	Monetization & Skins	All	IAP, косметика, LiveOps

3. High-Level Architecture





3.1 Слои

1. **Core.GameLogic** — движко-независимая бизнес-логика. // TODO: EXT_ENGINE_ABSTRACTIONS
2. **Platform Adapters** — тонкие обёртки под Unity, Godot, MonoGame, WPF.
3. **Presentation/UI** — меню, HUD, FX.

3.2 Design Patterns (+почему)

- **Strategy** — смена алгоритмов движения (прямое, диагональ, прыжок).
- **State Machine** — экраны: Splash → MainMenu → Gameplay → Pause → GameOver.
- **Observer/EventBus** — реакция UI/Audio на события ядра.
- **Factory Method** — спавн Food/Power-ups.
- **Command** — абстракция ввода (клава/тач/геймпад).
- **Service Locator (ограниченно)** — регистрация общих сервисов (AudioMixer, SaveGame).

3.3 Clean-Code Guidelines

- SOLID, KISS, DRY, YAGNI.
- Читаемые имена, small classes, DI через Constructor.
- // TODO: REFACTOR_MARKER — метка для будущих рефакторингов и улучшений архитектуры.

3.4 Dependency Injection

- Используем DI для внедрения сервисов (GameLoop, CollisionService, AudioMixer, Logger) в Core.GameLogic.
- В Platform Adapters (Unity, Blazor) подключаем контейнеры (например, встроенный DI в Blazor или Zenject в Unity).
- Цель: гибкость, тестируемость, слабая связанность и лёгкая расширяемость.

4. Error Handling & Logging

- **Global Exception Handler** в Platform Adapters.
- **Serilog** (+Sinks: Console, File, Seq).
- **Crash Analytics**: App Center / Unity Cloud Diagnostics.

5. Toolchain & Environments

Категория	Инструмент	Причина
IDE	Visual Studio 2022, Rider, VS Code (+GitHub Copilot, Cursor)	C#, кросс-платформа, AI-помощь по коду

Категория	Инструмент	Причина
Game Engine	Unity LTS 2025.x	Быстрый prototyping, экспорт WebGL/Mobile
Build	.NET 8 SDK (LTS), .NET 10 SDK (preview, опционально), Unity CLI (Command Line Interface)	Скриптовая сборка
SCM	GitHub (+Git LFS, 1 GB free quota, 1 GB traffic/month; для больших ассетов — отдельное хранилище)	Облако и History

6. UI / UX Design

Main Menu

- Весёлое, мультяшное оформление.
- Кнопки: Start Game, Settings, Exit.
- Анимация логотипа и фон с лёгкими эффектами переливания.

Settings

- Настройки громкости музыки и эффектов.
- Выбор темы (классика, музыка).
- Настройка размеров поля.

Pause Menu

- Resume, Restart, Exit to Main Menu.
- Полупрозрачный оверлей, чтобы видно было игру на фоне.

HUD (Heads-Up Display)

- Счёт игрока.
- Текущая длина змейки.
- Индикатор активного Power-up.
- Минималистичные иконки, мультяшный стиль.

Visual Style

- Мультяшная графика с тенями и мягкими анимациями.
- Яркие цвета, плавные переходы.
- Эффекты переливания для динамики и «кайфа».